COSC204
Computer Systems

Structs and Unions

Overview

- Last Lecture
  - Arrays, Malloc, and Free
- This Lecture
  - Structs and Unions
  - Source
    - K&R:
      - Brian Kernighan and Dennis Ritchie (1988), The C Programming Language. 2nd Edition. Prentice Hall. ISBN:0-13-110362-8
- Next Lecture
  - Abstract data types, pointers to routines, objects

Structs (not classes)

- C has an class-like *composite data type* called `struct`
- There are no methods
- There are no setters or getters
- There are no constructors
- There are no default values
- Everything is public
- Other than that, think of a `struct` as a `class`
- Or is it more like an `interface`? Or `record`?

Declaration

- Structs are declared exactly as you might expect

```
struct [<name>]
{
<type> <name>;
…
<type> <name>;
} [<variable>];
```

- For example,

```
struct point_2d
{
double x;
double y;
} point;
```

Typical Declaration

- Typically, an *anonymous struct* is declared and then given a name with a `typedef`
  - So that the type can be referred to using a shorter name
    - `point_2d` **vs** `struct point_2d`

- For example,

```
typedef struct
{
double x;
double y;
} point_2d;
point_2d location;
```

Struct Initialisation

- There are two ways to initialize a struct

```
typedef struct
{
double x;
double y;
} point_2d;
point_2d location;
```

- *Initializers*

```
point_2d location = {0.0, 1.0};
```

- *Designated initializers*

```
point_2d location = {.y = 1.0, .x = 0.0};
```

Accessing Data Members

- To access a member of a struct use the *member access operator, '.'*
  - Just like Java

```
typedef struct
{
double x;
double y;
} point_2d;

point 2_d location;

location.x = 0;
location.y = 1;
```

Shallow Copy

- Struct assignment is by *shallow copy*
  - Copy the members including pointers
    - But ***not*** the things pointers point to

```
typedef struct
{
float value;
uint8_t *string;
} thing;

void thing_print(thing object)
{
printf("%f ", object.value);
printf("%s\n", object.string);
}
```

Shallow Copy

```
int main(void)
{
thing first;
thing second;

first.value = 10.5;
first.string = malloc(6);
strcpy(first.string, "World");

second = first;

thing_print(first);
thing_print(second);
return 0;
}
```

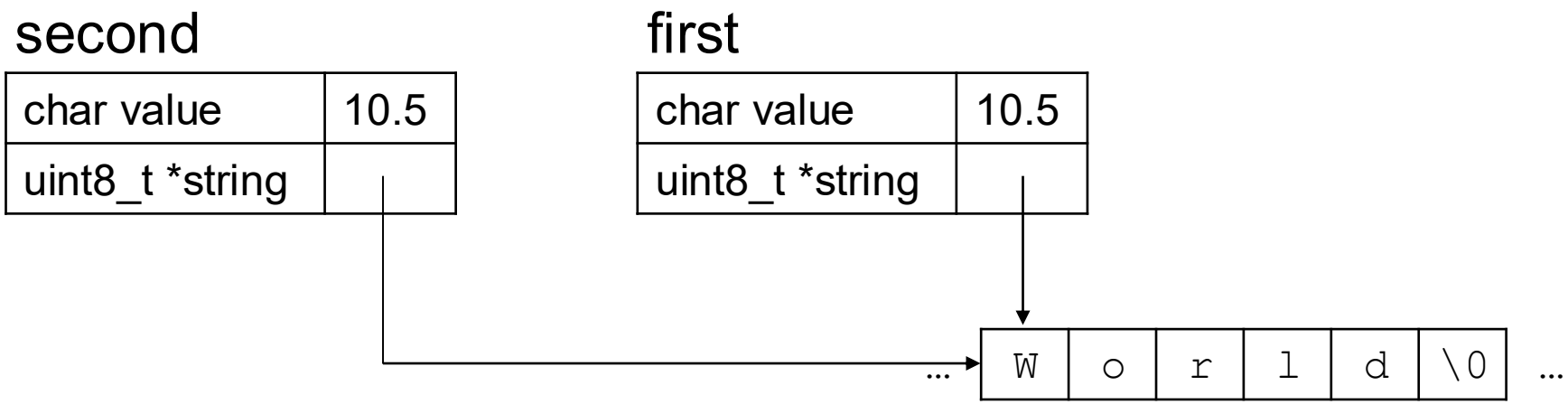
Shallow Copy

```
void thing_print(thing object)
{
printf("%f ", object.value);
printf("%s\n", object.string);
}
```

Shallow Copy

```
typedef struct
{
float value;
uint8_t *string;
} thing;

second = first;
```



Call by Value

- Structs are passed to routines *by value*
  - Different from Java classes, more like Java integers

```
void thing_print(thing object)
{
printf("%f ", object.value);
printf("%s\n", object.string);
object.value = object.value - 1;
}


…
thing_print(first);
…
```

This ***does not*** affect `first` because `object` is a ***copy*** of `first`

Call by Address

- To pass by address, take the address of the object and pass that as a pointer
  - But now we must *dereference* the pointer to turn it into an object
    - `(*object).value` we have syntax for that: `object->value`

```
void thing_print(thing *object)
{
printf("%f ", object->value);
printf("%s\n", object->string);
object->value = object->value - 1;
}
…
thing_print(&first);
…
```

This ***does*** affect `first` because `object` is a ***pointer*** to `first`

Returning Structs

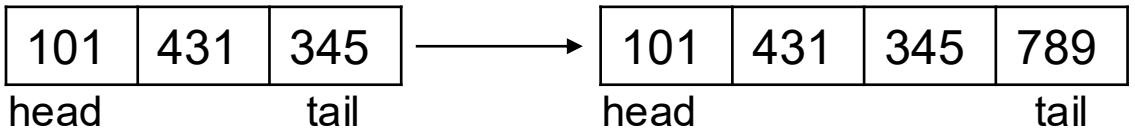
- Yes, return by value
- Yes, return a pointer to a struct

- ***DO NOT*** allocate a `struct` on the stack (as a local variable) and later `return` a pointer to it. That creates a ***dangling pointer***
  - C won't stop you from doing this
  - This is (inevitably) a bug
  - This is a difficult to find and fix

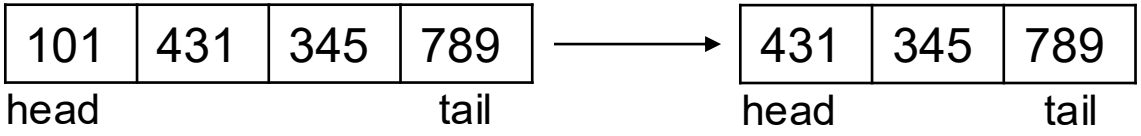
Queues

- A queue:
  - Enqueue to the tail
  - Dequeue from the head
  - First In First Out (FIFO)

enqueue(789)

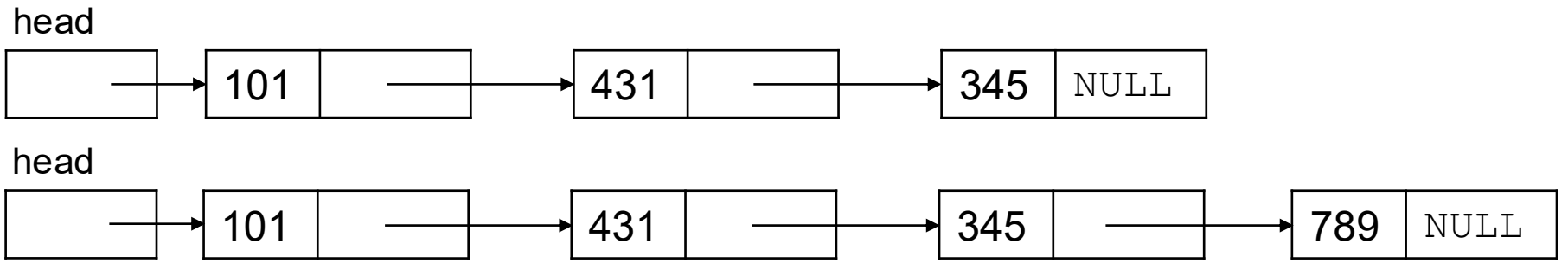


x = dequeue();



Enqueue Using Pointers

- We can represent a queue as a *linked list* of objects
  - We use `NULL` as a *sentinel* at the end of the list



- What we did:
  - Run down the list until we find `NULL`
  - Create a new object (value = 789, pointer = `NULL`)
  - Change the `NULL` in final object to the new object

Declarations

- A queue item has a value and a pointer

```
typedef struct q_item
{
uint32_t value;
struct q_item *next;
} queue_item;
```

- For convenience, declare a struct for the queue

```
typedef struct
{
queue_item *head;
} queue;
```

Constructor

- For convenience, let's have a *constructor*

```
queue *queue_new(void)
{
queue *this = malloc(sizeof(queue));
this->head = NULL;
return this;
}
```

head

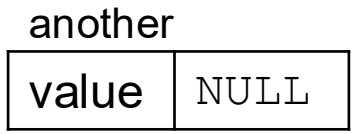


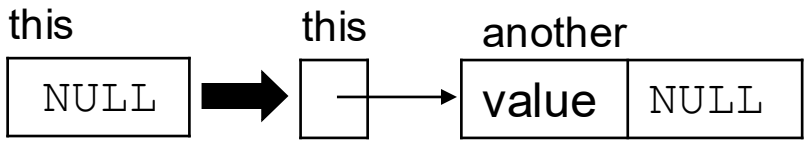
Enqueue

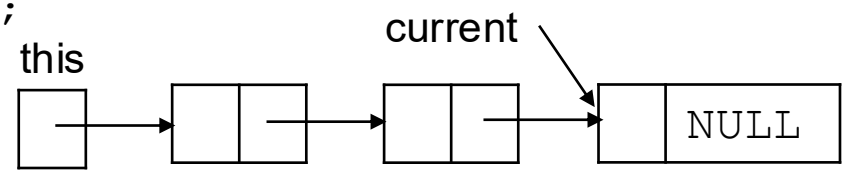
```
void queue_enqueue(queue *this, uint32_t value)
{
queue_item *another = malloc(sizeof(queue_item));
another->value = value;
another->next = NULL;

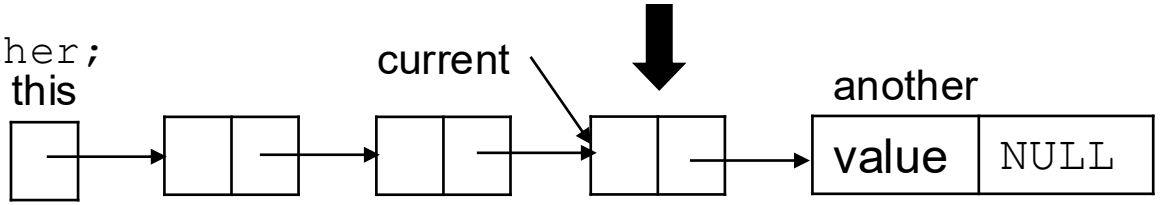
if (this->head == NULL)
   this->head = another;
else
   {
   queue_item *current = this->head;
   while (current->next != NULL)
      current = current->next;

   current->next = another;
   }
}
```



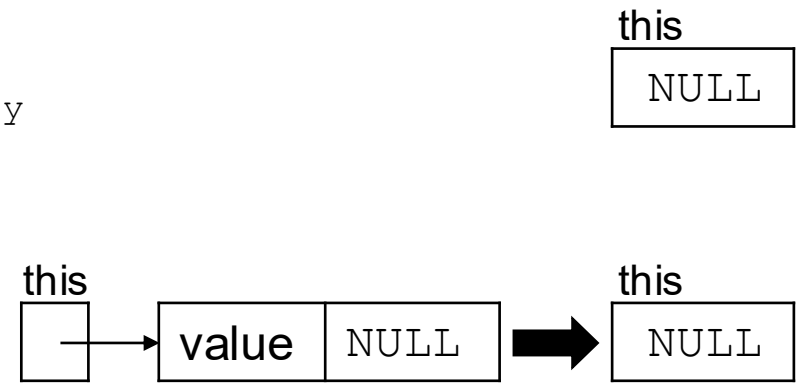






Dequeue

```
int64_t queue_dequeue(queue *this)
{
int64_t answer;
if (this->head == NULL)
   answer = -1;    // the queue is empty
else if (this->head->next == NULL)
   {
   answer = this->head->value;
   free(this->head);
   this->head = NULL;
   }
else
   {
   answer = this->head->value;
   queue_item *remainder = this->head->next;
   free(this->head);
   this->head = remainder;
   }
return answer;
}
```

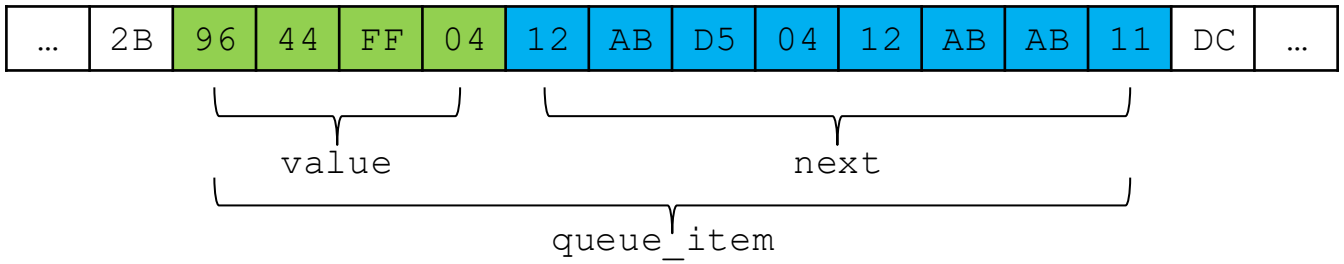




Structs as Views

- Structs are views of memory

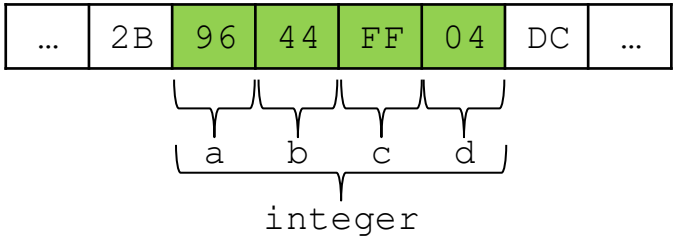
```
typedef struct q_item
{
uint32_t value;              // 4-byte integer
struct q_item *next;         // 8-byte pointer
} queue_item;
```



Unions

- Unions give us multiple views of the same memory
  - Each “line” in the declaration is a different view
  - Here we have 2 views
    - 4 x uint8_t
    - 1 x uint32_t

```
typedef union
{
struct
        {
        uint8_t a, b, c, d;
        };
uint32_t integer;
}  two_views;
```



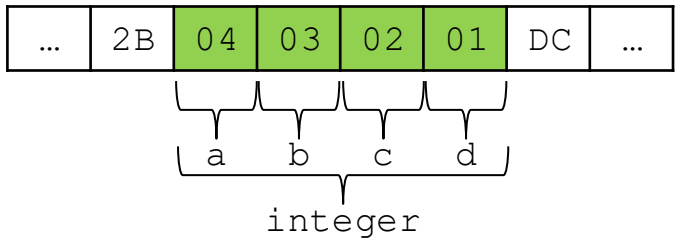
Unions

- What does this program do?
  - Why do programmers find themselves writing it?

```
int main(void)
{
two_views number = {.integer = 0x01020304};

printf("int   : %u\n", number.integer);
printf("bytes:%d %d %d %d\n", number.a, number.b, number.c, number.d);

return 0;
}
```



Homework

- What is deep copy?
- Write a routine to deep copy the queue
- Is the computer you are using big-endian or little-endian?
- Find out about the C language feature called bit fields
  - https://www.tutorialspoint.com/cprogramming/c_bit_fields.htm